

Lab 02

Creating a grid. deal.II step-1, step-2.

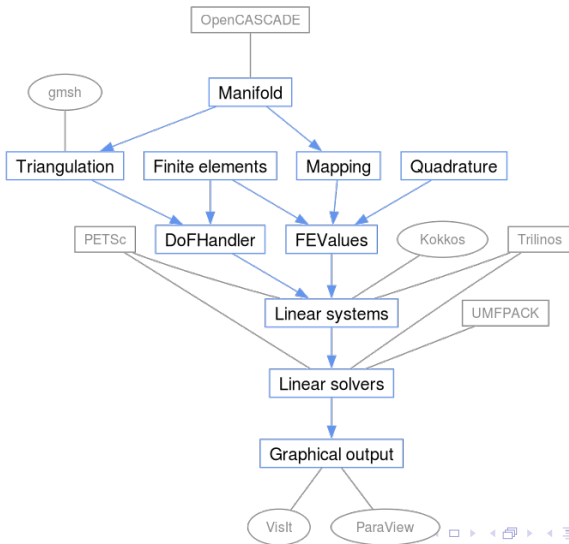
Advanced FEM Techniques - SISSA, 2025-26

Ankur Ankur

31 March 2026

Introduction

Primary groups of classes in deal.II interact



Triangulation group (Grids and Triangulations)

- ▶ Collection of cells and their lower-dimensional boundary objects.
- ▶ Stores geometry (vertices) and topology (connectivity)
- ▶ Does NOT know finite elements or DoFs

Creating the Triangulation (How to get the intuition)

```
Triangulation<dim, spacedim> tria;
```

Generating a Grid

```
GridGenerator::hyper_cube(tria, 0, 1);  
  
GridGenerator::hyper_shell(triangulation, center,  
                           inner_radius,  
                           outer_radius,  
                           10);
```

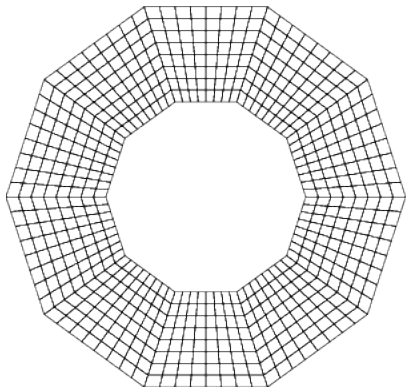
Manifold Concept (1/3)

- ▶ Controls placement of new points during refinement
- ▶ Without manifolds (Default): FlatManifold (straight edges)
- ▶ Without manifolds: Curved domains become distorted
- ▶ With manifolds: Geometry is preserved, better mesh quality

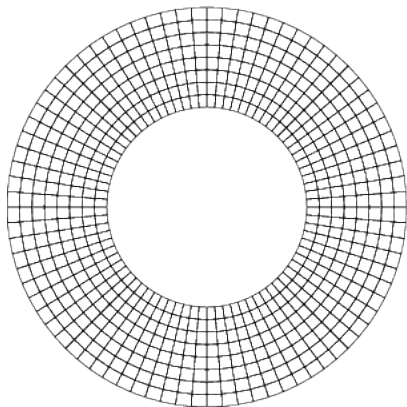
Example 1: Circular domain

```
const Point<2> center (1,0);  
const double inner_radius = 0.5,  
             outer_radius = 1.0;  
Triangulation<2> triangulation;  
GridGenerator::hyper_shell (triangulation,  
                             center, inner_radius,  
                             outer_radius,  
                             10);  
triangulation.reset_all_manifolds();  
triangulation.refine_global (3);
```

Manifold Concept (2/3)



Example 1



Example 2

Manifold Concept (3/3) : Example 2: Circular domain

```
#include <deal.II/grid/manifold_lib.h> //Remember!

const Point<2> center (1,0);
const SphericalManifold<2> manifold(center);
const double inner_radius = 0.5,
             outer_radius = 1.0;
Triangulation<2> triangulation;

GridGenerator::hyper_shell (triangulation,
                           center, inner_radius,
                           outer_radius,
                           10);

triangulation.reset_all_manifolds();
triangulation.set_all_manifold_ids(0);
triangulation.set_manifold (0, manifold);
triangulation.refine_global (3);
```

Step-1: Creating and Refining a Mesh

```
#include <deal.II/grid/grid_generator.h>
#include <deal.II/grid/grid_out.h>
#include <deal.II/grid/tria.h>
#include <iostream>
using namespace dealii;

int
main()
{
    Triangulation<2> tria;
    Point<2> center(0, 0);
    double inner_radius = 0.5, outer_radius = 1;
    GridGenerator::hyper_shell(tria, center,
                              inner_radius, outer_radius, 5);

    // Refinement.
    for (unsigned step = 0; step < 5; ++step)           //for loop 5 steps
    {
        for (auto &cell : tria.active_cell_iterators()) //range-based for loop
        {
            for (auto v : cell->vertex_indices())        // vertex indices, say numbering
            {
                Point<2> vertex = cell->vertex(v);        //actual coordinates of the vertex

                const double distance_from_center = vertex.distance(center);

                if (std::fabs(distance_from_center - inner_radius) <
                    1e-6 * inner_radius)
                {
                    cell->set_refine_flag();                // merked those cells
                    break;
                }
            }
        }
    }
}
```

```

        }
    }
    }
    tria.execute_coarsening_and_refinement();
}

```

Output the mesh (different formats)

```

GridOut grid_out;
grid_out.write_msh(tria, "first_mesh.msh"); //remember with the extension .msh
grid_out.write_mesh_per_processor_as_vtu(tria, "mesh"); // without the extension .vtu

// grid_out.write_svg(tria, "first_mesh.svg"); // not allowed with string, see
//documentation (only allowed with std::ofstream)

std::ofstream out("mesh_svg.svg"); // must provide an output stream i.e an std::ofstream
grid_out.write_svg(tria, out);      //header file #include <fstream>

return 0;
}

```


Step-2: Degrees of freedom and sparsity pattern

Introduction (Key Points):

- ▶ Enumeration of degrees of freedom is an entirely separate thing from the indices we use for vertices (think of higher-order elements).
- ▶ `DoFHandler` class enumerates all DoFs and provides their location on cells
- ▶ FEM matrices are sparse: only overlapping DoFs (on same or neighboring cells) produce nonzero entries.
- ▶ Sparsity allows efficient storage $O(N)$ and matrix-vector multiplications $O(N)$, enabling large-scale problems.
- ▶ DoFs can be renumbered to optimize sparsity patterns for certain solvers and preconditioners. For example, preconditioners such as SSOR will work better.

Step-2: Sparsity Patterns in deal.II

Two Types of Sparsity Patterns:

- ▶ **DynamicSparsityPattern (dsp):**
 - ▶ Flexible, growable structure but slow.
 - ▶ Used when building the sparsity pattern dynamically from the mesh.
 - ▶ Allows adding nonzero entries without knowing total count in advance.
 - ▶ Analogous to a fixed-size array: fast access and low memory overhead.
- ▶ **SparsityPattern (sp):**
 - ▶ Fixed, memory-efficient and fast.
 - ▶ Cannot add new entries after creation.
 - ▶ Used for initializing actual `SparseMatrix`.
 - ▶ Analogous to a fixed-size array: fast access and low memory overhead.

Step-2: Only the updated code from step-1

```
#include <deal.II/fe/fe_q.h>
#include <deal.II/dofs/dof_handler.h>
#include <deal.II/dofs/dof_renumbering.h>
#include <deal.II/dofs/dof_tools.h>
#include <deal.II/lac/sparse_matrix.h> //lac : Linear Algebra Components
#include <deal.II/lac/dynamic_sparsity_pattern.h> // for a intermediate sparsity pattern

int main() { ....
....   GridOut grid_out;
       grid_out.write_msh(tria, "first_mesh.msh");

       FE_Q<2> fe(1); // degree 1 in 2D.
       DoFHandler<2> dof_handler(tria); //initialized with the mesh (tria) as input.
       dof_handler.distribute_dofs(fe); //assigns global indices to DoFs

       // DoFRenumbering::Cuthill_McKee(dof_handler); //Cuthill_McKee (function)
       DynamicSparsityPattern dsp(dof_handler.n_dofs(), dof_handler.n_dofs());
       DoFTools::make_sparsity_pattern(dof_handler, dsp);
       SparsityPattern sp;
       sp.copy_from(dsp);

       std::ofstream out("sparsity-pattern_new.svg");
       sp.print_svg(out);

       SparseMatrix<double> system_matrix; //sets up the matrix structure but not fills values.
       system_matrix.reinit(sp);
       Vector<double> vector; //size equals the total number of DoFs
       vector.reinit(dof_handler.n_dofs());

       return 0;
}
```